

What Is an Exception in Java?

- An exception in Java is an event that disrupts the normal flow of a program.
- It occurs when something unexpected happens at runtime, such as dividing a number by zero, using an invalid array index, or accessing a missing file.
- When an exception occurs, Java creates an exception object and throws it.
- The normal execution stops immediately and Java searches for a matching catch block to handle the exception.
- If no matching catch block is found, the program terminates and Java prints the exception details on the console.
- An exception represents an abnormal situation that interrupts normal execution.

How are exceptions handled in Java, or explain the exception-handling mechanism in Java?

What Is Exception Handling in Java?

- Java handles runtime errors using a structured exception-handling mechanism that separates error-handling logic from normal program flow.
- When an error occurs, Java creates an exception object and throws it.
- Normal execution stops, and Java begins searching for a matching catch block.
- If a suitable catch block is found, the exception is handled; otherwise, the program terminates with an error message.
- This mechanism allows programs to handle unexpected situations without crashing.

Components of Exception Handling

- The try block contains code that may throw an exception.
- The catch block handles the exception thrown inside the try block.
- The finally block runs whether an exception occurs or not.
- The throw keyword is used to manually throw an exception.
- The throws keyword indicates that a method may throw an exception and must be handled by the caller.

Checked vs Unchecked Exceptions

- Java exceptions are divided into checked and unchecked categories.
- Checked exceptions must be handled using try–catch or declared with throws.
- Examples of checked exceptions include IOException and SQLException.
- Unchecked exceptions extend RuntimeException and do not require explicit handling.
- Examples include ArithmeticException, NullPointerException, and IndexOutOfBoundsException.

Using the throws Keyword

- When a method may throw a checked exception, it must declare it using the throws keyword.
- For example, a method reading a file may declare “throws IOException.”
- The caller must either handle the exception using try–catch or propagate it further with its own throws declaration.
- This approach ensures that checked exceptions are properly handled or safely propagated.

Final Interview Summary

- Java uses exceptions to handle abnormal situations in a controlled manner.
- The try block contains risky operations, and the catch block handles failures.
- The finally block always runs to release resources or perform cleanup.
- The throw keyword manually throws an exception, and the throws keyword declares that a method might throw one.
- Java supports both checked and unchecked exceptions, ensuring safety and reliability in applications.
-

How Can You Catch Multiple Exceptions in Java?

- Java allows you to catch multiple exceptions using two approaches: writing separate catch blocks for each exception type, or using a single catch block that handles several exception types using the pipe symbol.
- When an exception occurs inside the try block, Java stops normal execution, creates an exception object, and searches for a matching catch block.
- If Java finds a matching catch block, it handles the exception; if not, the program terminates.
- **The main goal of multi-exception handling is to make the program stable when different types of errors may occur in the same block of code.**
-

What Is Exception Chaining in Java?

What Is Exception Chaining?

- Exception Chaining is a mechanism in Java that allows one exception to wrap another exception.
- When a low-level technical exception occurs, you can catch it and throw a new, more meaningful business-level exception.
- The original exception is attached as the cause, so Java preserves the full error path.

- This technique helps developers understand what failed at the business level and why it failed at the technical level.
- Java supports exception chaining using the “cause” parameter available in all Throwable classes.

Why Exception Chaining Is Useful

- Exception chaining allows applications to throw a clean, readable business message without exposing low-level technical errors directly.
- At the same time, it preserves the original exception so developers can debug the real root cause.
- It provides clear, layered error information: the top-level failure and the underlying exception.
- It improves debugging, error tracking, and overall maintainability of large applications.
-

Can You Throw Any Exception Inside a Lambda Expression’s Body?

- Yes, you can throw exceptions inside a lambda expression, but the lambda must follow the exception rules of the functional interface it is assigned to.
- If the functional interface method does not declare a checked exception, then the lambda cannot throw that checked exception unless it is handled inside the lambda.
- Unchecked exceptions like RuntimeException can be thrown freely because they do not require a throws clause.
- Checked exceptions like IOException can only be thrown if the functional interface method declares them; otherwise Java gives a compile-time error until the exception is handled inside the lambda.

How to Create a Custom Exception in Java

- A custom exception in Java is a user-defined exception that extends either `Exception` or `RuntimeException`.
- You create one when built-in exceptions do not clearly describe your business error.
- Extending `Exception` creates a **checked exception** that forces handling.
- Extending `RuntimeException` creates an **unchecked exception** that does not require try-catch.
- The custom exception simply needs a constructor that passes a message to the parent class.